

# Canon Layer Exploration Report

Kirill Zemlianskii

June 12, 2026

## Abstract

abstract

## 1 Data: Synthetic Benchmarks

### 1.1 Task Descriptions

**Depo.** Depo [?] is a synthetic reasoning benchmark that tests multi-hop knowledge retrieval. Each training instance encodes  $n$  directed edges of a circular permutation followed by one or more queries. A query specifies a start node  $q$  and a hop distance  $k$ ; the model must predict the  $k$ -th successor without intermediate chain-of-thought steps. Formally, each sequence has the form

$$\langle \text{bos} \rangle x_1 y_1 \cdots x_n y_n \langle \text{query}_k \rangle q a \langle \text{eos} \rangle,$$

where  $y_i = f^k(x_i)$  for the ground-truth permutation  $f$ . The model is trained only on the answer tokens  $a$ . The benchmark admits two variants: Depo1 uses 1–2-token nodes (base vocabulary 50,  $K=8$ ) and Depo2 uses 5–7-token nodes (base vocabulary 3,  $K=16$ ).

**Brevo.** Brevo [?] evaluates *reasoning breadth*: the ability to process multiple dependencies simultaneously. Each instance encodes a directed acyclic graph (DAG) of at most  $N$  nodes, each of in-degree at most 4, followed by a query vertex  $q$ :

$$\langle \text{bos} \rangle x_1 y_1 \cdots x_m y_m \langle \text{query} \rangle q \langle \text{ans} \rangle a_1 a_2 \cdots a_p \langle \text{eos} \rangle.$$

The model must output all vertices recursively reachable from  $q$ , sorted in topological order from the leaves inward. Because valid topological orderings are non-unique, the model must mentally compute the *full* topological order of the DAG before generating the first answer token, making this a demanding breadth-first reasoning task [?]. Two variants are used: Brevo1 (single-token vertex names,  $N \in \{70, 90, 110\}$ , sequences up to 1024 tokens) and Brevo2 (2–4-token vertex names,  $N \in \{30, 40, 50\}$ , sequences up to 1536 tokens).

### 1.2 Stability Across Seeds

A central question is whether these benchmarks produce reliable, reproducible training curves. We trained the same model architecture multiple times with different random seeds and compared the resulting loss trajectories.

**Text modeling is stable.** As a reference, Figure ?? shows two Llama and two Canon seeds on the standard FineWeb-Edu text modeling task. The curves are nearly indistinguishable within each model class, confirming that the observed variance on synthetic tasks is not caused by general optimizer noise.

**Depo exhibits large seed-to-seed variance.** Figures ?? and ?? show Depo training for an 8-layer/512-dim model and a 12-layer/768-dim Canon model respectively.

The 8-layer model training shows that the benchmark outcome flip depending solely on random initialization. The 12-layer result illustrates a *grokking* regime [?]: one seed never solves the task while the other eventually solves it after a prolonged plateau. Figure ?? shows the same effect when loss is computed over all tokens rather than answer tokens only.

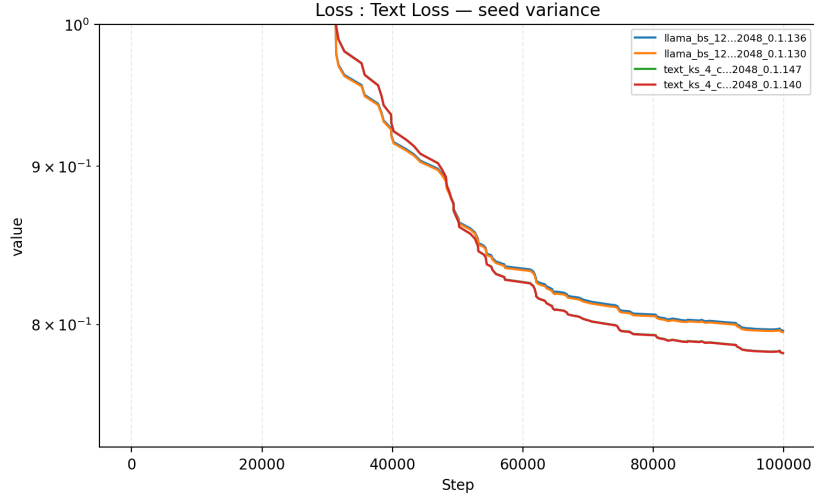


Figure 1: Training loss for two Llama seeds and two Canon seeds on text modeling. Curves within each model class overlap, demonstrating negligible seed-to-seed variance.

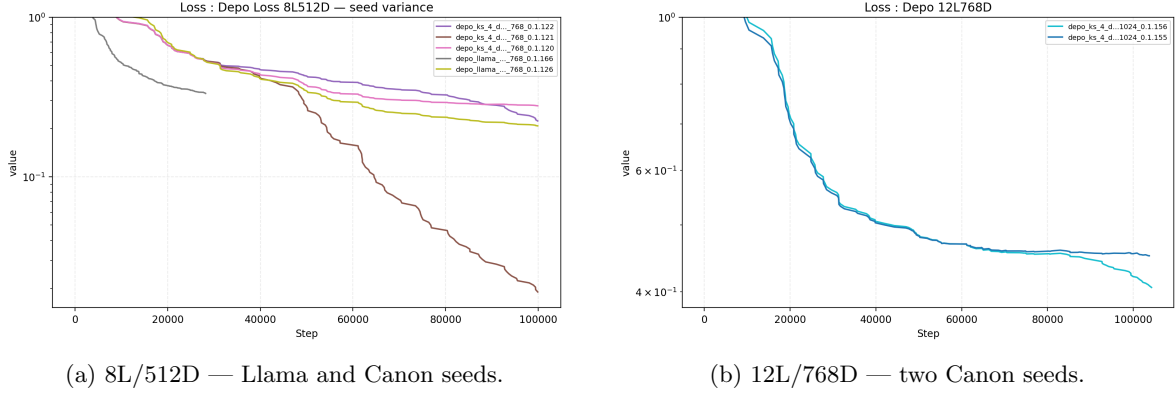


Figure 2: Depo training loss across seeds. In the 8L/512D setting one Llama seed converges dramatically faster than all other runs. In the 12L/768D setting one Canon seed stagnates throughout while the other undergoes a sudden grokking event, dropping sharply after many steps of little progress.

**Brevo exhibits the same instability.** Figure ?? shows three Brevo seeds for a 110-node single-node configuration. All three runs diverge from one another, with no sign of convergence toward a common loss value.

### 1.3 Discussion

Both Depo and Brevo can in principle identify a better-performing architecture when results are averaged over many seeds. However, the high seed-to-seed variance makes individual training runs unreliable proxies for studying *training dynamics*. Metrics such as gradient norms, representation scales, and activation kurtosis fluctuate as much across seeds as they do across architectures, preventing meaningful comparison on a single run. For dynamics analysis, a stable task such as standard text modeling is therefore preferable.

## 2 Canon Layers

### 2.1 Background

A canon layer is a learned linear convolution applied to the last  $k$  hidden-state vectors along the sequence dimension. Let  $\mathbf{h}_t$  be the hidden state at position  $t$  and  $k$  be the kernel size. The canon

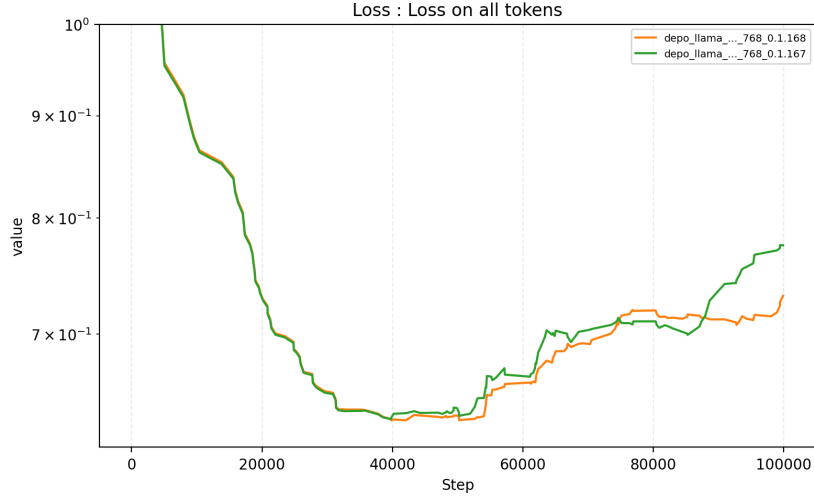


Figure 3: Depo loss computed on all tokens (including context), confirming the same high variance observed on answer tokens alone.

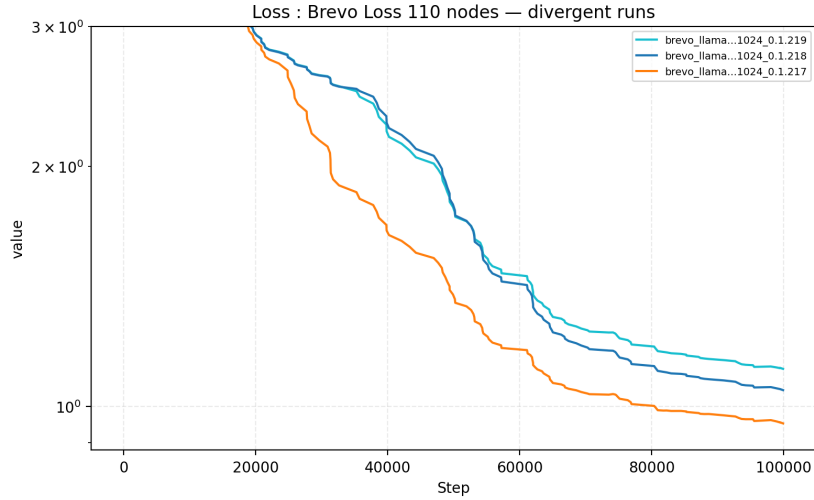


Figure 4: Brevo training loss for three random seeds. Losses diverge across seeds, mirroring the instability observed on Depo.

output at position  $t$  is

$$\tilde{\mathbf{h}}_t = \sum_{i=0}^{k-1} w_i \mathbf{h}_{t-i},$$

where  $\{w_i\}$  are scalar learned weights shared across the feature dimension. Canon layers can be inserted at four positions within each transformer block:

- **A** — before the self-attention sublayer (pre-attention),
- **B** — after the self-attention sublayer (post-attention),
- **C** — before the feed-forward sublayer (pre-FFN),
- **D** — after the feed-forward sublayer (post-FFN).

The key hyperparameters are the kernel size  $k$ , the weight initialization scheme, and the subset of positions and layers in which canon is applied. Unless stated otherwise, experiments use Canon-ABCD (all four positions) across all transformer layers.

## 2.2 Performance vs. Llama Baseline

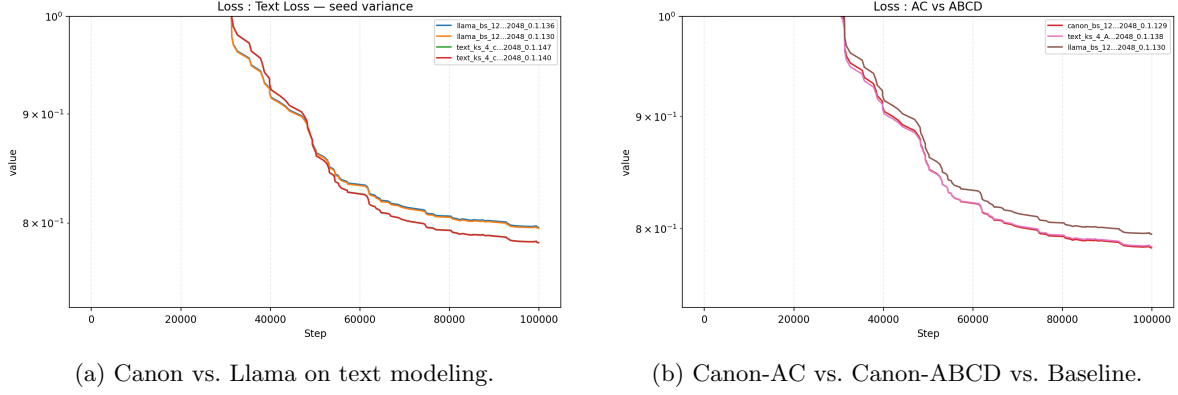


Figure 5: Left: Canon overtakes the Llama baseline around step 50 000 and reaches a final loss of 0.776 vs. 0.790. Right: Using only the pre-attention and pre-FFN canon positions (AC) captures nearly all of the gain over the baseline.

On the FineWeb-Edu text modeling task, Canon is slower to start than Llama but overtakes the baseline around step 50 000 and achieves a final loss of **0.776** compared to **0.790** for the Llama baseline (Figure ??).

A position ablation (Figure ??) shows that Canon-AC (positions A and C only) performs nearly as well as the full Canon-ABCD configuration. Adding post-attention (B) and post-FFN (D) canons provides only marginal additional benefit.

## 2.3 Ablations

### 2.3.1 Kernel Size

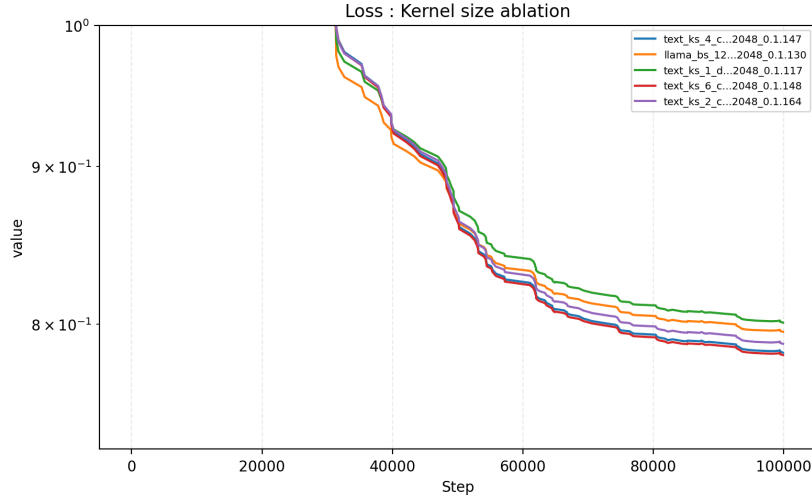


Figure 6: Training loss on text modeling for kernel sizes 1, 2, 4, 6, and the Llama baseline. Larger kernels consistently outperform smaller ones;  $k=1$  is even worse than no canon.

Figure ?? compares kernel sizes  $k \in \{1, 2, 4, 6\}$  against the Llama baseline. A kernel of size 1 reduces to a pointwise rescaling and performs worse than no canon at all, confirming that cross-token mixing is essential. Performance improves monotonically with kernel size up to  $k=6$ .

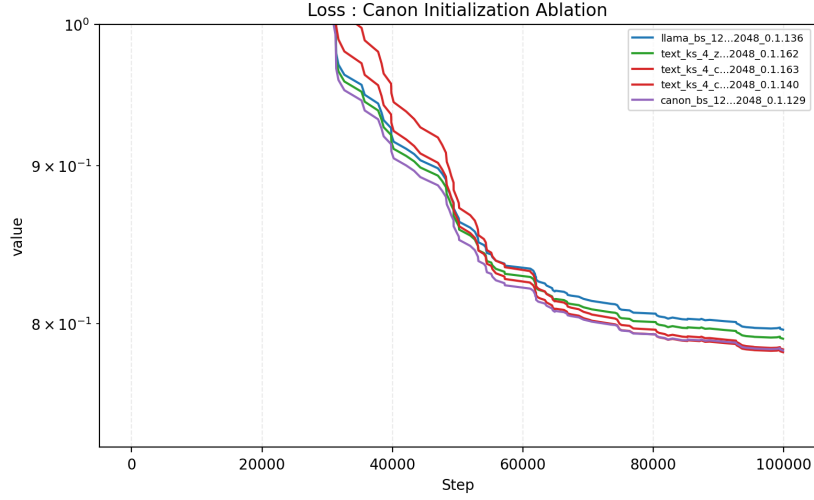


Figure 7: Initialization ablation on text modeling. All canon variants outperform the Llama baseline;  $1/k$  constant initialization is best.

### 2.3.2 Weight Initialization

We compare four initialization schemes for the canon weights  $\{w_i\}$  (Figure ??):

- `const_var_sqrt`: constant  $1/k$ ,
- `const_var`: constant  $1/\sqrt{k}$ ,
- `default_init`:  $\mathcal{U}[-1\sqrt{k}, 1\sqrt{k}]$ ,
- `zeros_init`: all zeros.

Notably, even zero initialization beats the Llama baseline, indicating that the architectural benefit of canon is not entirely dependent on initialization. The  $1/k$  scheme (equal weight per shift, normalized by kernel size) is the strongest.

### 2.3.3 Canon Position Within the Network

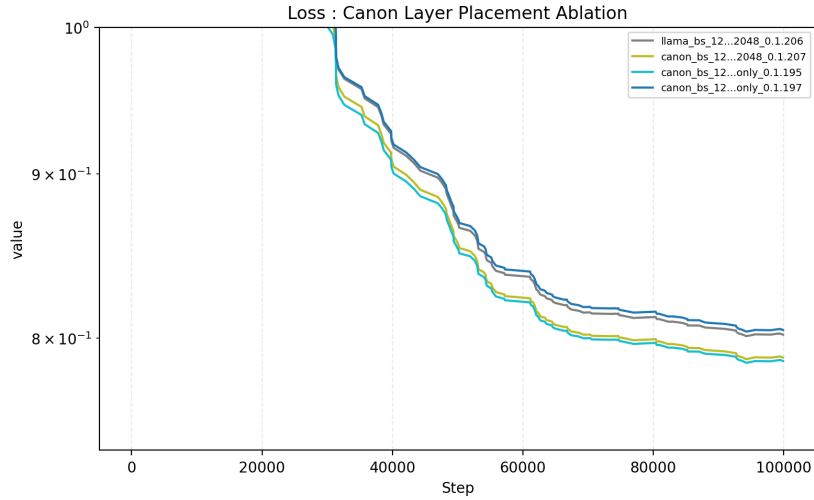


Figure 8: Loss for canon applied to all layers, the first layer only, the last layer only, and the Llama baseline.

Figure ?? compares inserting canon across all transformer layers versus restricting it to the first or last layer only. Placing canon in all layers provides the strongest benefit; single-layer variants still improve over the baseline but to a lesser degree.

### 2.3.4 Normalization Type

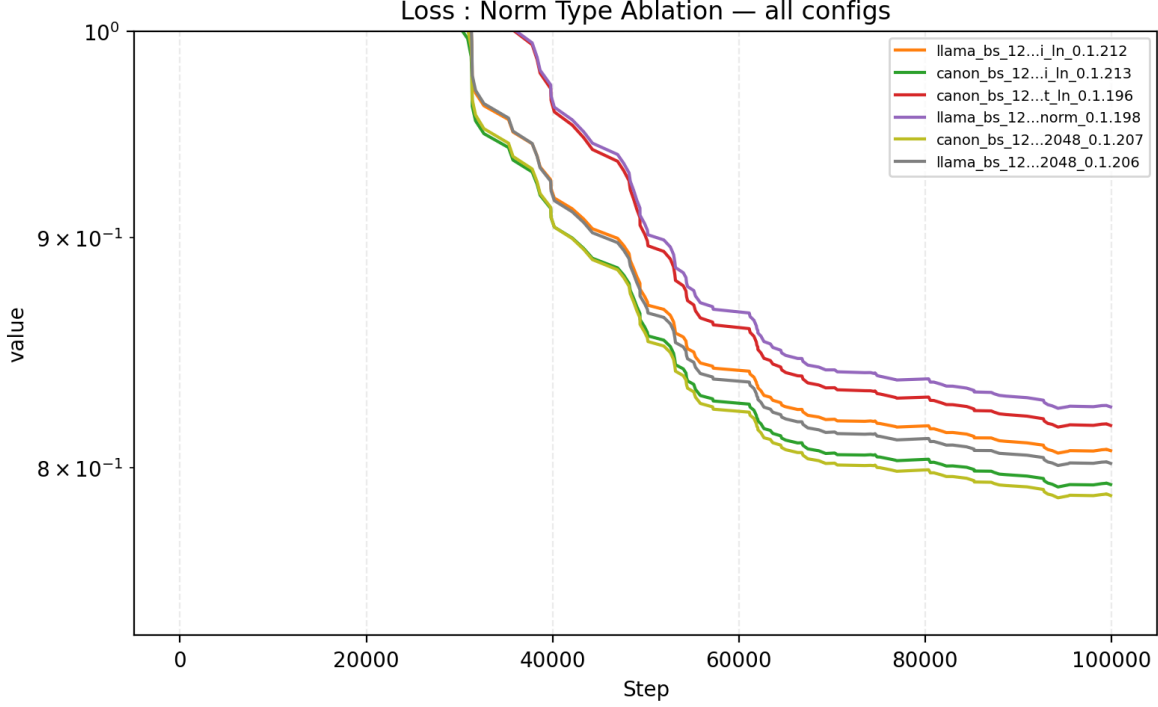


Figure 9: Training loss for Canon and Llama under Pre-LN, Post-LN, and Peri-LN normalization. Canon consistently outperforms the same-normalization Llama baseline across all three schemes.

Canon’s benefit is not specific to a single normalization scheme. Figure ?? shows that Canon outperforms the corresponding Llama baseline under Pre-LN, Post-LN, and Peri-LN normalization. Figures ?? and ?? show the associated gradient and representation-scale dynamics per normalization type.

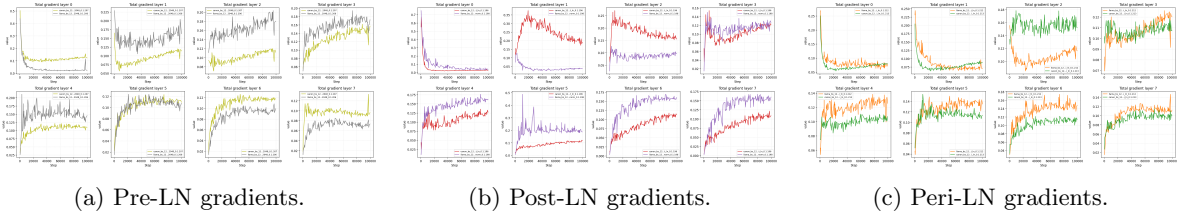


Figure 10: Per-layer gradient contributions for Canon and Llama under each normalization type.

## 2.4 Training Dynamics

All dynamics figures below compare a Canon-ABCD model ( $\mathbf{ks}=4$ , default initialization) against the Llama baseline on the text modeling task.

### 2.4.1 Canon Weights: Farther Shifts Receive Larger Weights

Figure ?? shows the learned scalar weights  $\{w_i\}$  for each canon type and a representative set of layers. Across all types and layers, weights corresponding to larger shifts (e.g.,  $i=3$ ) are larger than those for

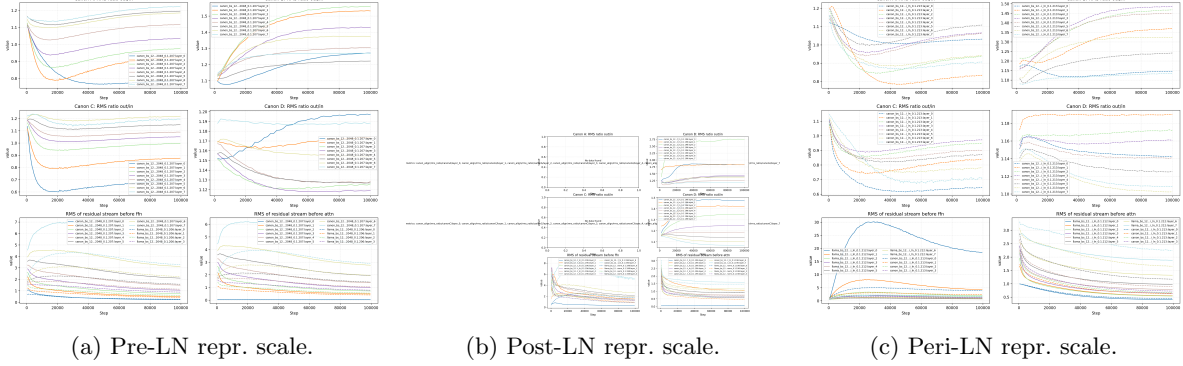


Figure 11: Residual stream RMS scale per layer for Canon and Llama under each normalization type.

smaller shifts (e.g.,  $i=1$ ). This suggests the model actively learns to integrate context from distant positions rather than relying on the immediately preceding token.

### 2.4.2 Representation Scaling

Figure ?? shows the ratio of the output RMS to the input RMS for each canon type. All canon types amplify the magnitude of representations (ratio  $> 1$ ). The effect is strongest in deeper layers (closer to the LM head), particularly for types A and C. This depth-dependent amplification may explain why Canon gains advantage over the Llama baseline later in training: as the residual stream grows at depth, the canon contribution compounds.

### 2.4.3 Gradient Uniformity

A well-known challenge in deep network training is gradient imbalance: gradients in early layers are much smaller than in later layers, slowing learning. Figure ?? quantifies this for both models. The Llama baseline shows a ratio of approximately  $20/6 \approx 3.3\times$  between the deepest and shallowest layer gradients. Canon reduces this to approximately  $11/8 \approx 1.4\times$ , indicating that canon layers substantially alleviate gradient vanishing in early layers.

### 2.4.4 Outlier Features

Outlier features (activations with extreme values) are a known source of training instability. Figure ?? shows activation kurtosis per layer, where high kurtosis signals the presence of outlier features. Canon dramatically reduces kurtosis across all layers compared to the Llama baseline, with the notable exception of layer 0, which retains elevated kurtosis in both models.

### 2.4.5 Cosine Similarity Between Canon Output and Input

Figure ?? measures the cosine similarity between a canon layer’s output and its input across training. A similarity close to 1 indicates the canon is acting as a near-identity (residual) transformation. Deeper layers show higher cosine similarity for all canon types, meaning the convolution contributes less mixing as depth increases. Canon types A and C exhibit stronger mixing (lower similarity) in the first layers, which is consistent with the representation scaling results showing that A and C have greater depth-dependent amplification.

## References

- [1] Z. Allen-Zhu and Y. Li. *Physics of Language Models: Part 4.1 — Learning Knowledge and Reasoning Skills*. arXiv preprint, 2023.
- [2] A. Power, Y. Burda, H. Edwards, I. Babuschkin, and V. Misra. *Grokking: Generalization Beyond Overfitting on Small Algorithmic Datasets*. arXiv:2201.02177, 2022.

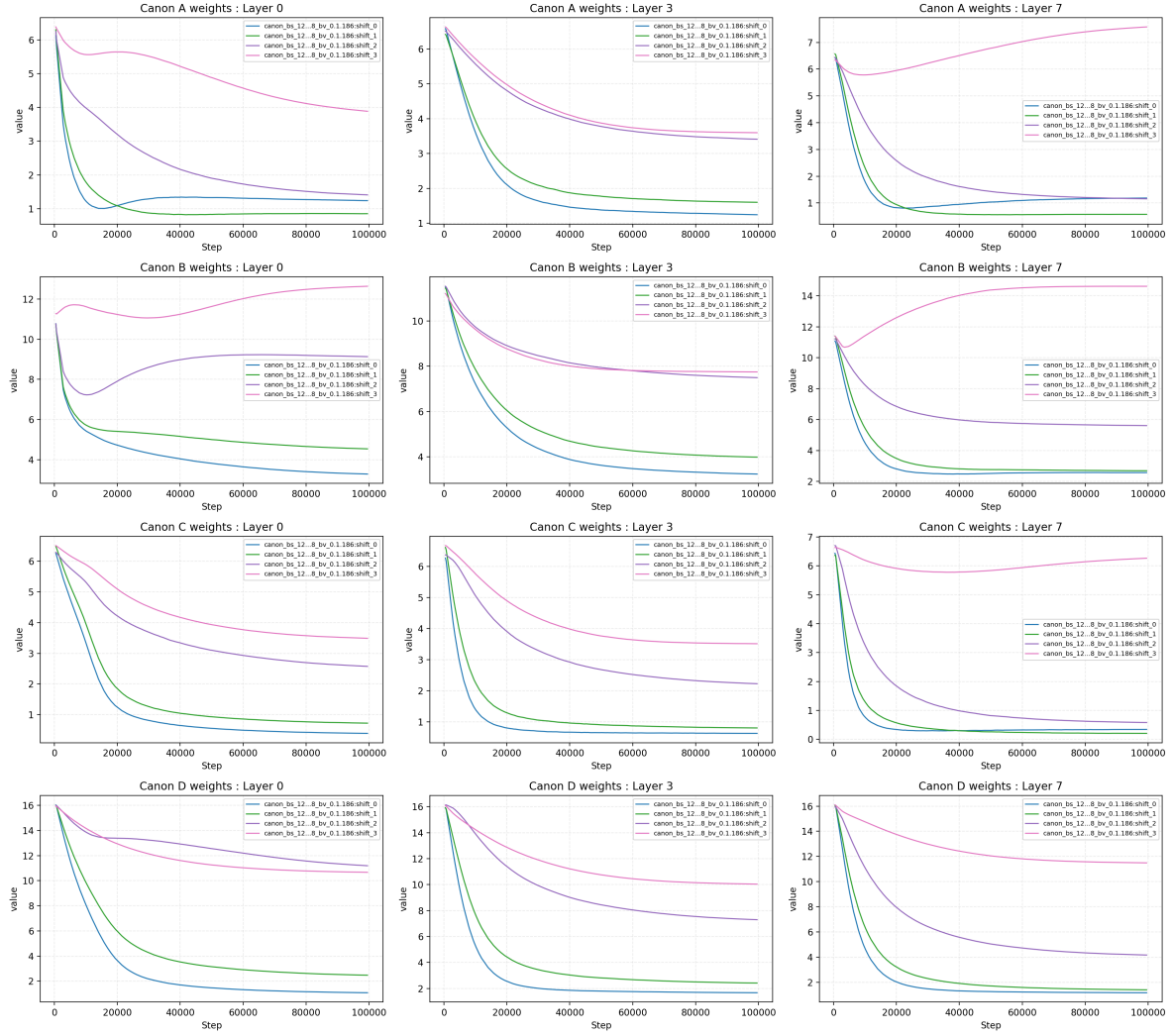


Figure 12: Learned canon weights for types A, B, C, D at layers 0, 3, and 7 throughout training. Weights for larger shifts (more distant tokens) are consistently larger than weights for shorter shifts.

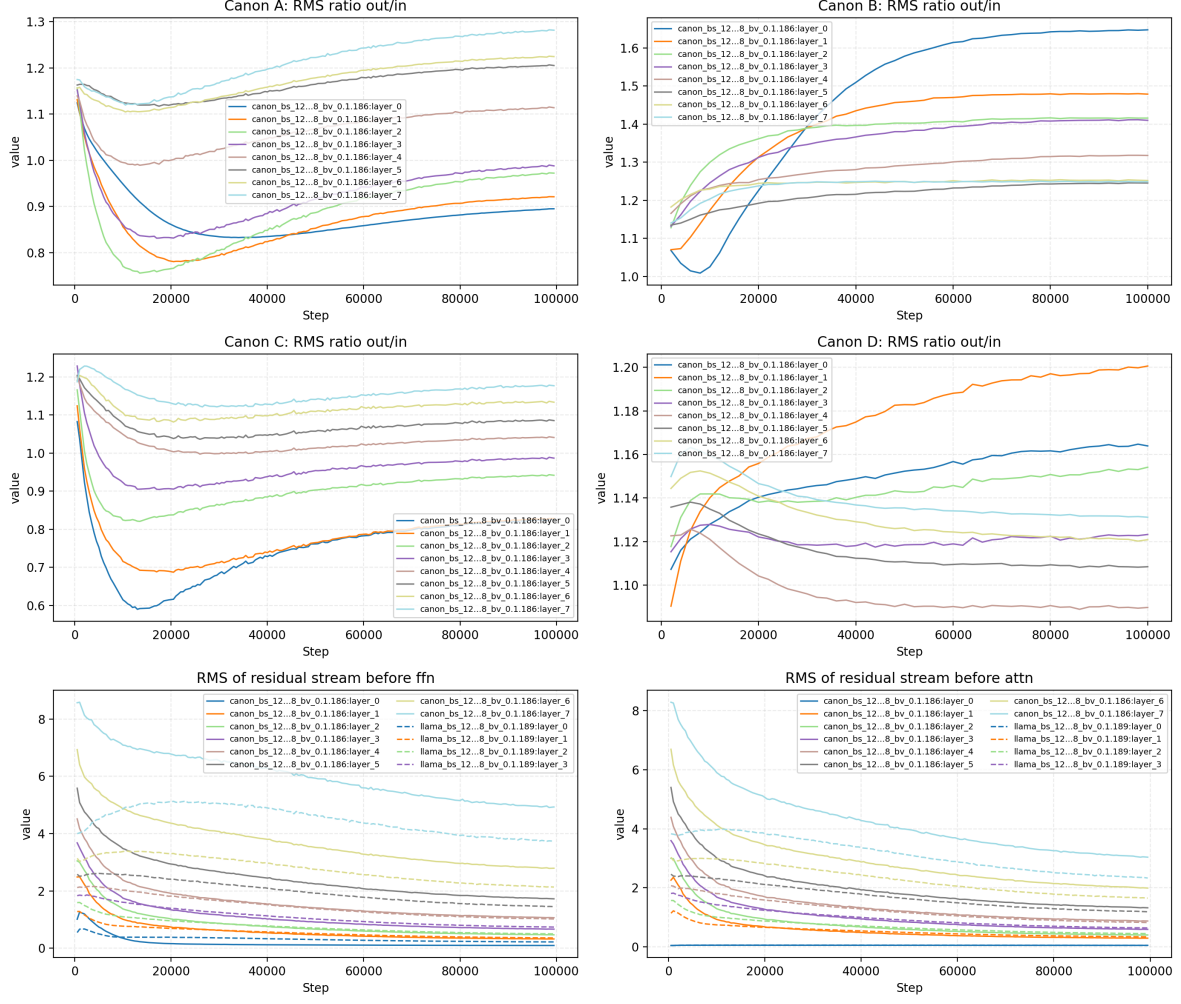


Figure 13: RMS ratio of canon output to input (top) and residual stream RMS before attention and FFN sublayers (bottom). Canon upscales representations, especially in deeper layers.

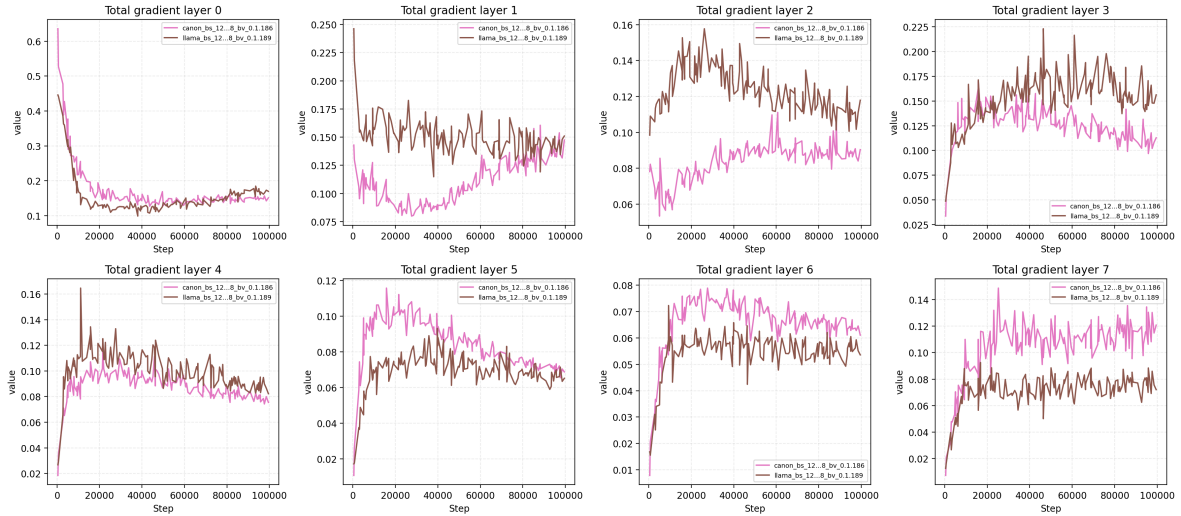


Figure 14: Per-layer gradient contribution magnitudes for Canon and Llama mid-training. The Llama baseline shows a steep decline from deep to shallow layers; Canon produces a much flatter profile.

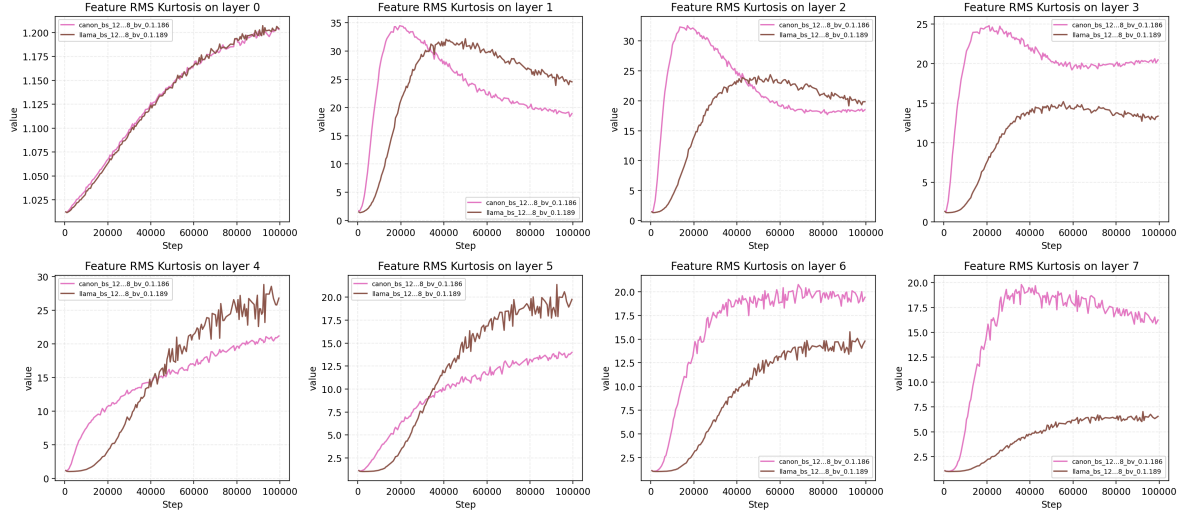


Figure 15: Activation kurtosis per layer for Canon and Llama. High kurtosis indicates outlier features. Canon greatly reduces kurtosis in all layers except layer 0.

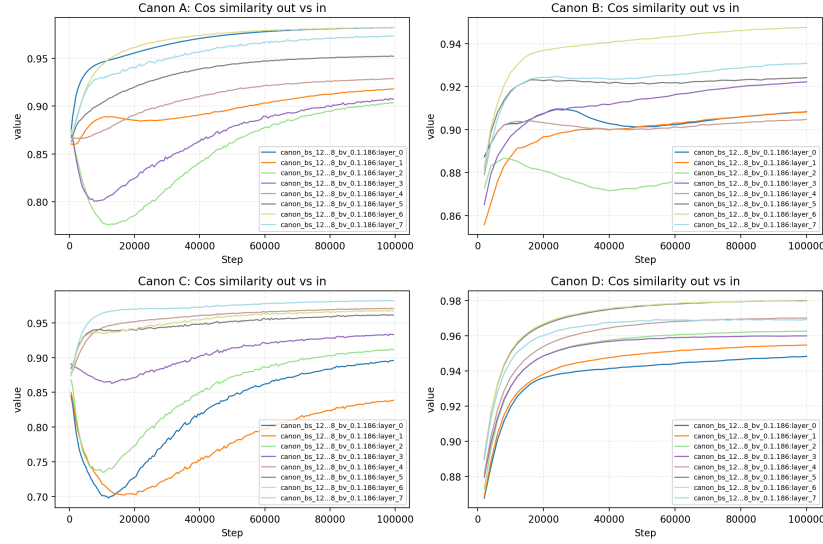


Figure 16: Cosine similarity between the canon output and its input for types A, B, C, D across layers. Lower similarity indicates stronger mixing. Canon types A and C mix more in early layers; all types converge toward identity-like behavior in deeper layers.